```
!pip install torch transformers datasets peft accelerate -q

import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer, TrainingArguments, Trainer
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
```

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
```

config.json: 100% 483/483 [00:00<00:00, 29.7kB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.98kB/s]

vocab.txt: 232k/? [00:00<00:00, 4.33MB/s]

tokenizer.json: 466k/? [00:00<00:00, 4.61MB/s]

model.safetensors: 100% 268M/268M [00:05<00:00, 88.3MB/s]

Loading weights: 100% 100/100 [00:00<00:00, 392.32it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]

```
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
pre_classifier.bias      | MISSING    |
pre_classifier.weight    | MISSING    |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```
dataset = load_dataset("imdb")

def tokenize_function(example):
    return tokenizer(example["text"], truncation=True, padding="max_length")

tokenized_dataset = dataset.map(tokenize_function, batched=True)

train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(200))
```

README.md: 7.81k/? [00:00<00:00, 369kB/s]

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 106MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:01<00:00, 68.6MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 82988.69 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 86630.07 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 119699.19 examples/s]

Map: 100% 25000/25000 [00:31<00:00, 1151.33 examples/s]

Map: 100% 25000/25000 [00:18<00:00, 1303.12 examples/s]

Map: 100% 50000/50000 [00:39<00:00, 1362.03 examples/s]

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_lin", "v_lin"],
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)
```

```
peft_model = get_peft_model(model, lora_config)
peft_model.print_trainable_parameters()
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no"
)

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned me
  super().__init__(loader)
```

[63/63 14:48, Epoch 1/1]

Step	Training Loss
10	0.705481
20	0.666515
30	0.675843
40	0.639409
50	0.642424
60	0.628055

```
TrainOutput(global_step=63, training_loss=0.6577299776531401, metrics={'train_runtime': 914.0524, 'train_samples_per_second': 0.547, 'train_steps_per_second': 0.069,
'total_flos': 67369703424000.0, 'train_loss': 0.6577299776531401, 'epoch': 1.0})
```

```
trainer.evaluate(eval_dataset=test_dataset)

text = "The movie was fantastic and inspiring!"
inputs = tokenizer(text, return_tensors="pt")

outputs = peft_model(**inputs)
prediction = torch.argmax(outputs.logits, dim=1)

print("Predicted label:", prediction.item())
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned me
  super().__init__(loader)
```

[25/25 02:13]

```
Predicted label: 1
```